COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 16, 2026	**DATE SUBMITTED:**	September 16, 2026
NAME:	Jaime Luis M. Demain	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Seatwork 9.1 **Implementing Trees**

1. PROCEDURE OUTPUT

What procedures did you perform?

1. Code Snippet

```
#include <iostream>
#include "binaryTree.h"

int main(){

    //creating the root node:
    BinaryTree<char>* root = new BinaryTree<char>('A');

    //creatomg a child of root node A:
    root->insertLeftChild('B');
    root->insertRightChild('C');

    //insert a child to node B:
    root->getLeftChild()->insertLeftChild('D');
    root->getLeftChild()->insertLeftChild('F');

    //insert a child to node C:
    root->getRightChild()->insertLeftChild('E');

    //print the root node:
    std::cout<< "Root: " << root->getRootVal()<<std::endl;

    //children of the root node
```

```
    std::cout << "root left child: " <<
root->getLeftChild()->getRootVal() << std::endl;
    std::cout << "root right child: " <<
root->getRightChild()->getRootVal() << std::endl;

    //children of the sub trees B and C
    std::cout << "node B left child: " <<
root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout << "node B right child: " <<
root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    root->getLeftChild()->getRightChild()->setRootVal('E');
    std::cout << "node B right child: " <<
root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    return 0;
}
```

Main.cpp

```
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree{
    private:
        T key;
        //pointers:
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;
    public:

        //constructor:
        BinaryTree(T rootObj);

        //adding/inserting a child node:
        void insertLeftChild(T rootObj);
        void insertRightChild(T rootObj);
```

```
        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        T getRootVal();
        T setRootVal();

};

template <typename T>
BinaryTree<T>:: BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}

template <typename T>
void BinaryTree<T>:: insertLeftChild(T newNode){

    if(leftChild == nullptr){
        leftChild = new BinaryTree<T>(newNode);
    }else{
        //CREATE A NEW NODE:
        BinaryTree<T> *newChild = new BinaryTree<T> (newNode);

        //move the existing left child below the new Node
        newChild->leftChild = leftChild;
        leftChild = newChild;
    }
}

template <typename T>
void BinaryTree<T>:: insertRightChild(T newNode){
    if(rightChild == nullptr){
       rightChild = new BinaryTree<T>(newNode);
    }else{
        //CREATE A NEW NODE:
        BinaryTree<T> *newChild = new BinaryTree<T> (newNode);
```

```
        //move the existing left child below the new Node
        newChild->rightChild = rightChild;
        rightChild = newChild;
    }

    }
}

template <typename T>
BinaryTree<T>* BinaryTree<T>::getLeftChild(){
    return leftChild;
}

template <typename T>
BinaryTree<T>* BinaryTree<T>::getRightChild(){
    return rightChild;
}

template<typename T>
T BinaryTree<T>::getRootVal(){
    return key;
}

template<typename T>
void BinaryTree<T>:: setRootVal(T obj){
    key = obj;
}

template<typename T>
bool BinaryTree<T>::isLeaf(){

    return leftChild == nullptr && rightChild == nullptr;
}

#endif
```

binaryTree.h

2. Documentation

The BinaryTree<T> class is a generic implementation of a binary tree where each node serves as the root for a separate subtree. It includes the value of the object as well as pointers to left and right subtrees of this node.

Class Attributes

- T key: Value that the current node contains.
- BinaryTree<T>* leftChild: Pointer to the root of the left subtree (nullptr if this subtree is absent).
- BinaryTree<T>* rightChild: Pointer to the root of the right subtree (nullptr if this subtree is absent).

Methods & Member Functions

BinaryTree<T>(T rootObj) **(Constructor)**

Description: Creates a new node with a certain value.
Operation: Sets key to rootObj and initializes leftChild and rightChild to nullptr.

void insertLeftChild(T newNode)

Description: Insert a new node having newNode as data in left side of the current node.
Operation:

- If the leftChild is null, it creates a new BinaryTree<T> instance and assigns it to it.
- If there is a left child, then pushes the existing subtree down the tree by setting the left child of new node to be leftChild and setting leftChild to be the new node.

void insertRightChild(T newNode)

Description: Insert a new node having newNode as data in right side of the current node.
Operation:

- If rightChild is null, then create a new BinaryTree<T> instance and assign it directly.
- If there is a right child, then splice the new node in the right branch of the tree by making the existing right child the child of the new node.

BinaryTree<T>* getLeftChild()
Description: Accessor function to access the left child subtree of the current node.
Operation:It returns leftChild.

BinaryTree<T>* getRightChild()
Description: Accessor function for retrieving the pointer to the right child subtree.
Operation: Returns rightChild, allowing further programmatic traversal of the right subtree.

T getRootVal()
Description: Accessor function for retrieving the data payload contained within the current node.
Operation: Returns the value of key.

void setRootVal(T obj)
Description: Mutator function for updating the data payload contained within the current node.
Operation: Sets the value of key to obj without making any changes to the structure of the tree or any pointers to branches.

bool isLeaf()
Description: Evaluation function for determining if the current node is a leaf node.
Operation: Returns true if both leftChild and rightChild evaluate to nullptr; otherwise, returns false.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, Quillbot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.